PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description

Managing budgets across different life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. **PocketSmart AI** addresses this challenge through a **GenAI-powered, cross-platform recommendation system** that delivers personalized, budget-based suggestions for products and services.Using **Gemini 1.5 Flash Pro**, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as **Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO**, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets.PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine that bridges multiple categories and e-commerce ecosystems.

Team Members Name & Roles:

1.Name:Jeevajothi.P(Team Leader)

Role: Documentation

2.Name: Anu.S(Team Member)

Role: API Testing video

3.Name:Kirubalini.K(Team Member)

Role: Coding

4. Name: Hemawathy.R(Team Member)

Role: Voice over,Demo Video

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the **Home Budget Planner**. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quantities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon.Recommendations are balanced across functionality, style, and price, ensuring users can decorate their spaces beautifully without overspending.

Scenario 2: AI-Based Party Budget Planning

In the **Party Planner**, users input their total budget, guest count, event type, and venue details. Based on these inputs, the AI allocates the budget proportionally across **catering, decoration, and entertainment**, sourcing options from vendors listed on Swiggy, Zomato, and even accommodation services like OYO.The system tailors suggestions based on event type (e.g., birthday, corporate, wedding), helping users organize a successful event without financial guesswork.

Scenario 3: Jewelry Recommendations for Occasions

The **Jewelry Budget Planner** allows users to enter a budget, select the occasion, and define style preferences. Users can optionally upload an outfit image for the AI to consider color coordination and aesthetics. Gemini 1.5 Flash Pro analyzes the data and offers jewelry options from platforms like Amazon and Flipkart, tailored to occasion and style.This ensures that users receive elegant, matching jewelry suggestions that fit both their personal style and financial plan.

Architecture Overview

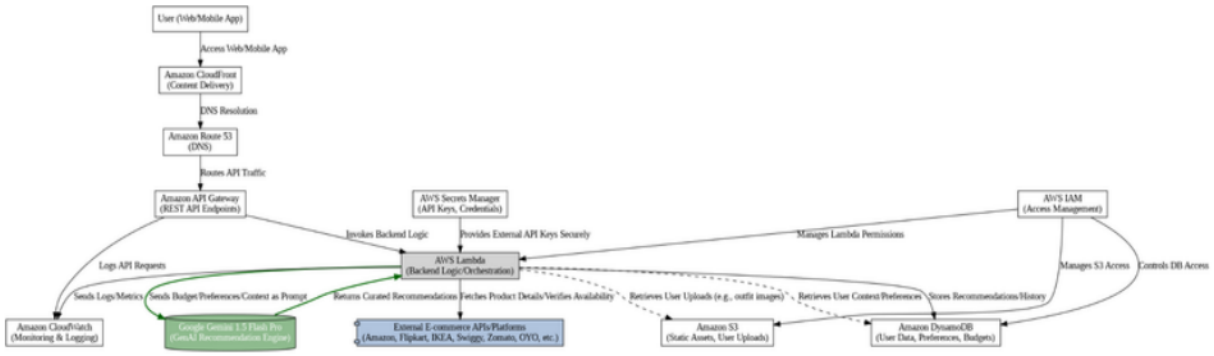
PocketSmart AI is an intelligent, modular platform designed to generate personalized, budget-aware recommendations across domains like home interiors, party planning, and jewelry selection. It integrates a lightweight Flask backend with Google's **Gemini 1.5 Flash Pro**, a powerful multimodal AI model that processes user inputs (text, numbers, and images) to deliver smart recommendations.

Core Technologies

- **Flask**: Backend service layer for request handling, routing, and session control.
- **Gemini 1.5 Flash Pro**: AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
- **Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.)**: Product and service data sourcing.
- **Frontend (HTML/CSS + JS)**: Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component	Description
Frontend UI	Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category.
Flask Backend	Manages user sessions, routes (`/generate-home`, `/generate-party`, `/generate-jewelry`), and handles communication between UI and Gemini AI.
Gemini AI Layer	Processes inputs using Gemini 1.5 Flash Pro to:



Pre-requisites

1. IBM Cloud Account Setup: IBM Cloud Console
2. Watsonx Access and Configuration: Watsonx Overview
3. Python Basics: Python.org
4. FastAPI Framework: FastAPI Docs

Project Workflow

1. Gemini AI Setup & Initialization

- **Activity 1.1**: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- **Activity 1.2**: Configure API keys, usage limits, and test integration for multimodal support (text + image).
- **Activity 1.3**: Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development

- **Activity 2.1**: Design Flask-based backend structure for routing, session control, and planner modules.
- **Activity 2.2**: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.
- **Activity 2.3**: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)

- **Activity 3.1**: Define Flask routes like `/generate-home`, `/generate-party`, `/generate-jewelry`.
- **Activity 3.2**: Organize backend using modular structure:
  - `routes/`: Endpoints
  - `services/`: Recommendation logic & AI calls
  - `models/`: Input/output schemas
- **Activity 3.3**: Setup CORS headers for frontend communication and implement basic session management.
- **Activity 3.4**: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development

- **Activity 4.1**: Build responsive HTML forms for:
  - Home Interior Planner (room details, quantities)
  - Party Planner (event type, budget, guests)
  - Jewelry Planner (budget, occasion, outfit image upload)
- **Activity 4.2**: Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization

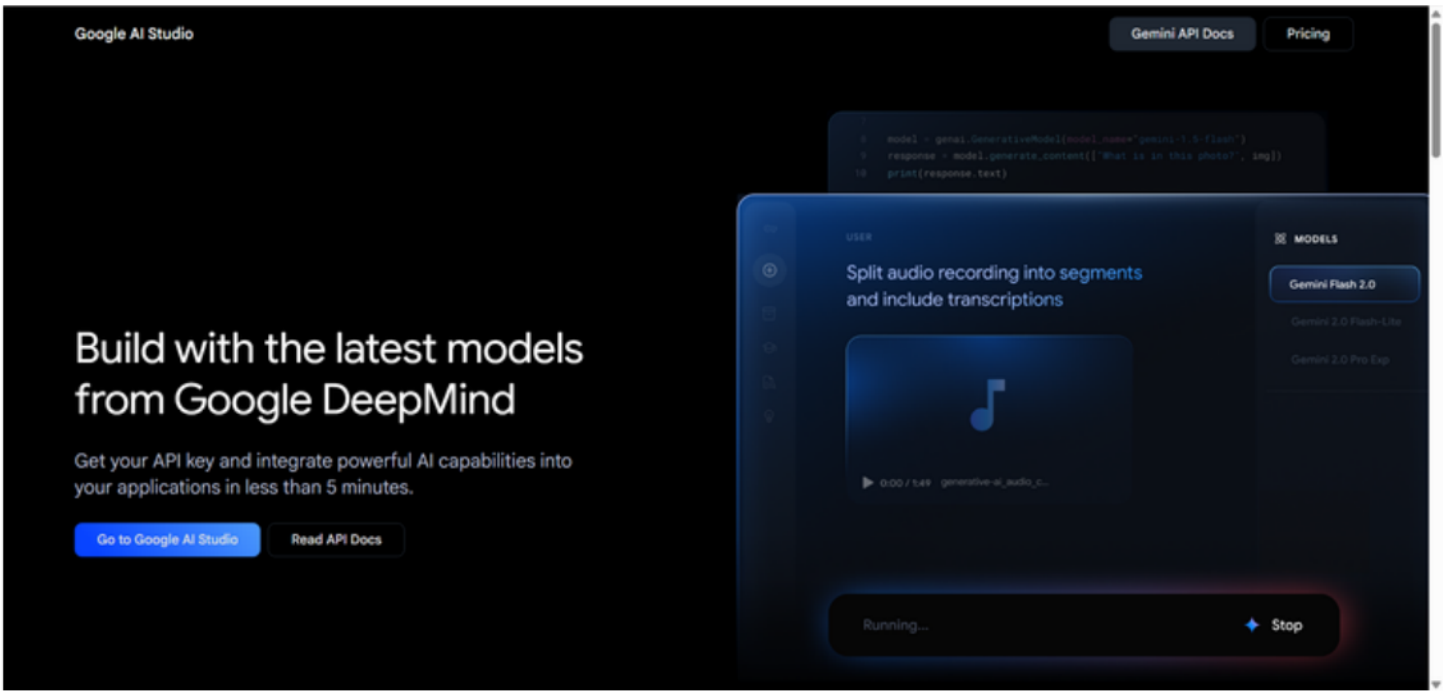
- **Activity 5.1**: Perform testing with real-world use cases across all three planners.
- **Activity 5.2**: Evaluate Gemini response quality, budget adherence, and platform accuracy.
- **Activity 5.3**: Optimize prompts and add validations to prevent input edge-case failures.
- **Activity 5.4**: Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization

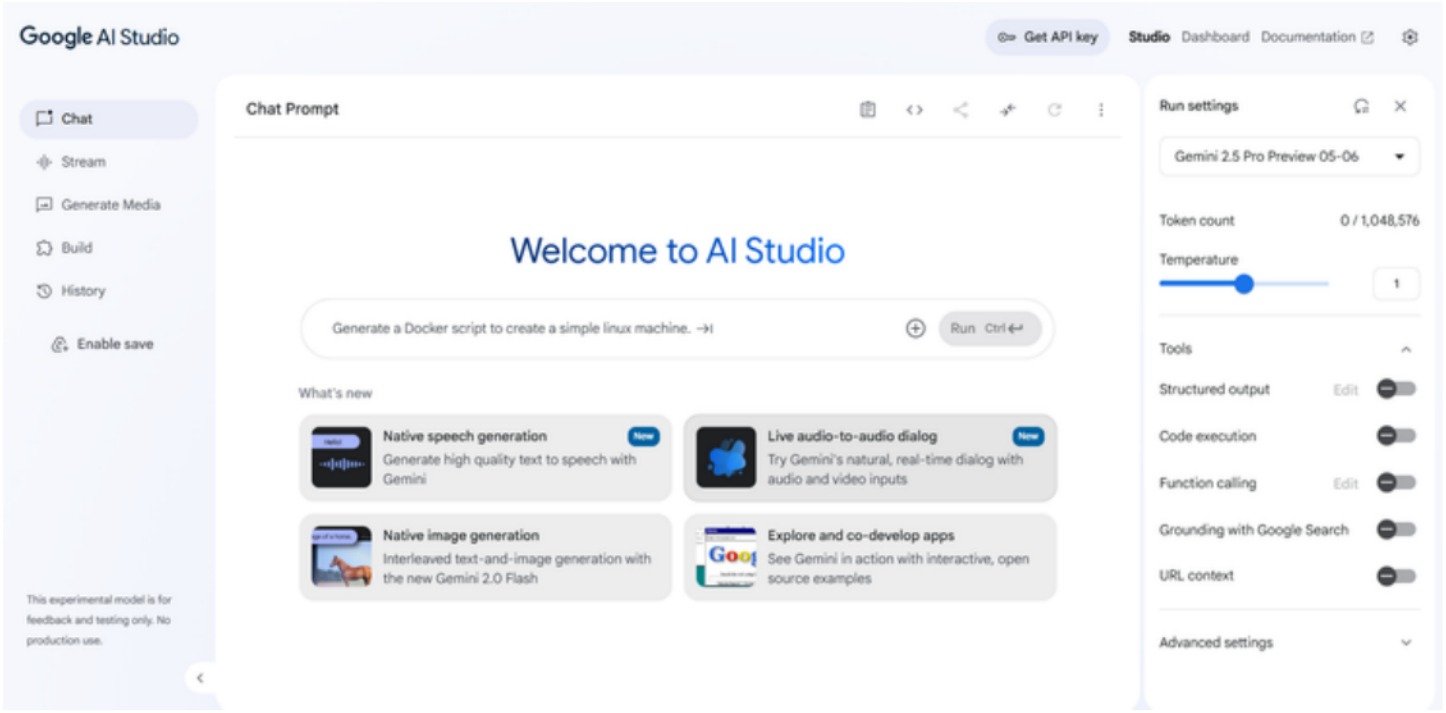
In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **PocketSmart AI**. The platform relies on **Gemini 1.5 Flash Pro**—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner).Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

- **Activity 1.1: Set up Google Cloud account and enable Gemini API**

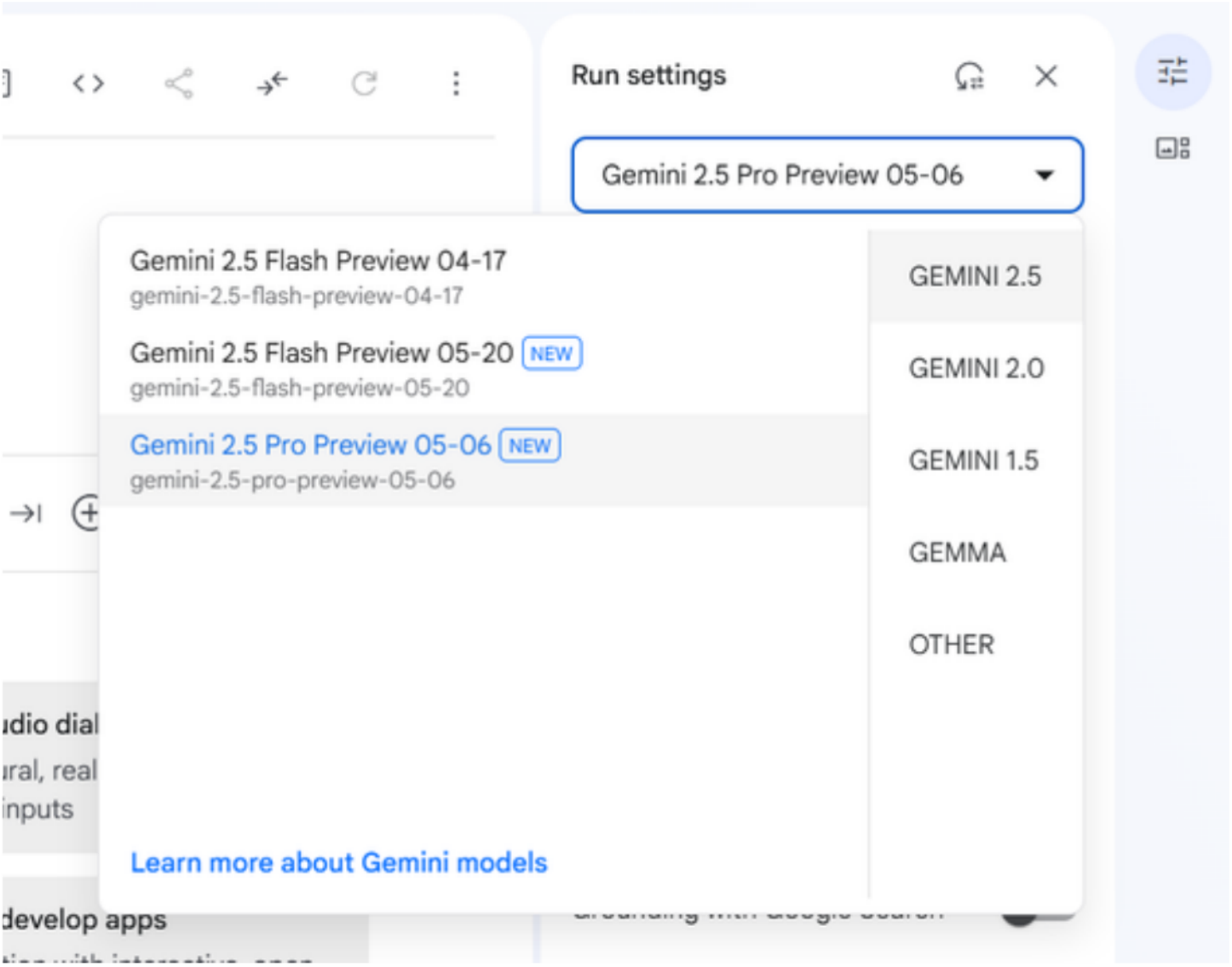
- Sign in or create a Google Cloud account via console.cloud.google.com.



- Accept all terms and conditions and click on Get API key



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

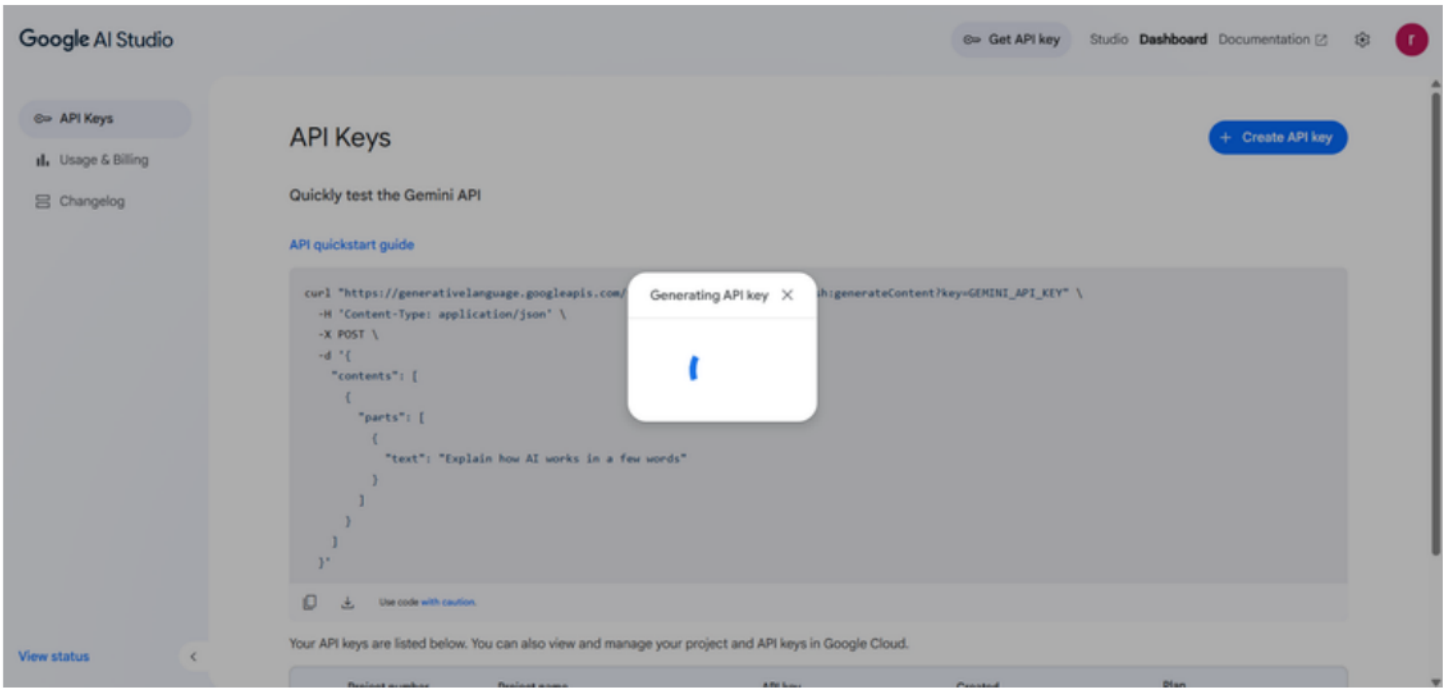


Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key



- Once click on API key you will the api key and make sure to copy the API key



- **Activity 1.3: Validate Gemini API connectivity**
  - Make a sample prompt call using Python to confirm integration.

app.py

```python
from google import genai
from google.genai import types
from PIL import Image
from io import BytesIO
import base64

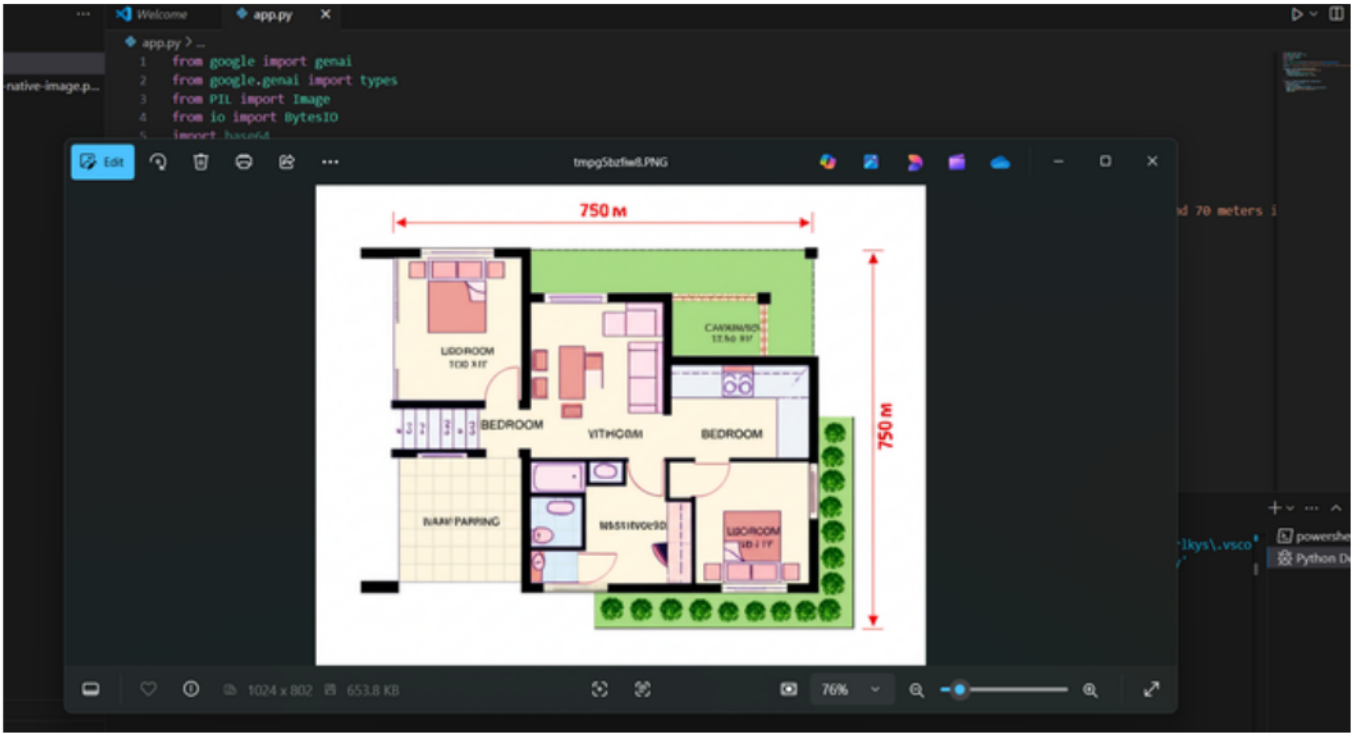
client = genai.Client(api_key="AIzaSyDdzTxAIisLXlQ")

contents = ('Generate a detailed 2D architectural floor plan for a modern two-story house with a total area of 150 meters in length and 70 meters

response = client.models.generate_content(
    model="gemini-2.0-flash-exp-image-generation",
    contents=contents,
    config=types.GenerateContentConfig(
      response_modalities=['Text', 'Image']
    )
)

for part in response.candidates[0].content.parts:
  if part.text is not None:
    print(part.text)
  elif part.inline_data is not None:
    image = Image.open(BytesIO((part.inline_data.data)))
    image.save('gemini-native-image.png')
    image.show()
```

  - Test both text-only and image + text prompts to ensure multimodal compatibility.



-

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations.The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (gemini_utils.py) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure